

LIBIF

Hệ thống Số hóa Thư viện & Quản lý Tài liệu Thông minh

PHƯƠNG PHÁP PHÁT TRIỂN PHẦN MỀM & HƯỚNG DẪN CI/CD

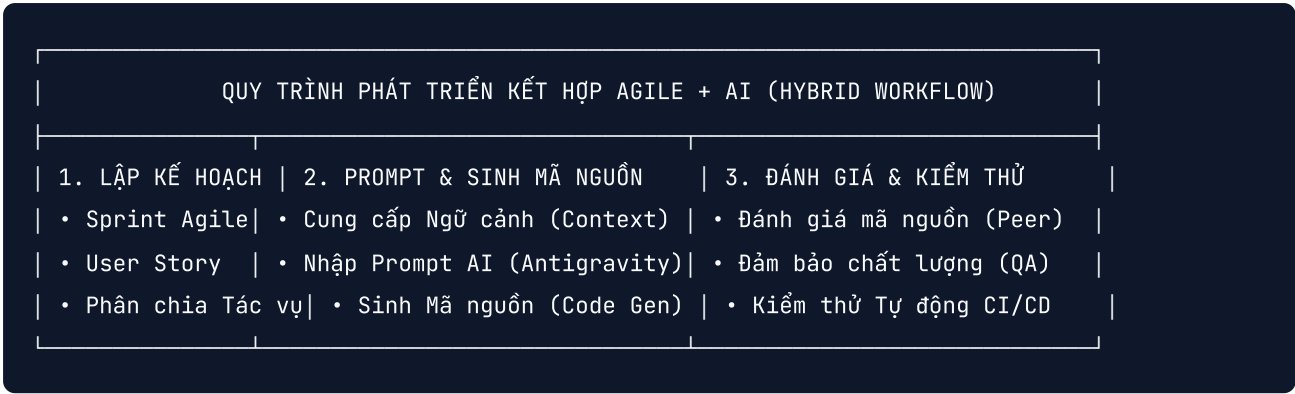
(PHƯƠNG PHÁP PHÁT TRIỂN VỚI SỰ HỖ TRỢ CỦA AI & HƯỚNG DẪN TRIỂN KHAI CI/CD)

Chi tiết về phương pháp phát triển phần mềm Agile hỗ trợ bởi AI (GitHub Copilot & Antigravity AI Assistant), các kịch bản sinh mã nguồn, quy trình kiểm thử và triển khai CI/CD container hóa tự động với Docker.

Trường	Nội dung
Tên dự án	LIBIF — Hệ thống Số hóa Thư viện Thông minh
Phiên bản	v1.0 (Phương pháp Phát triển & Kiến trúc CI/CD)
Ngày	20 tháng 07, 2026
Tác giả	Kỹ sư DevOps & Kỹ sư Trưởng Phần mềm

1. PHƯƠNG PHÁP AGILE HỖ TRỢ BỞI AI

Dự án LIBIF áp dụng quy trình **Agile / Scrum** trong khung thời gian **8 tuần** được tăng tốc bởi các Trợ lý Lập trình AI (**GitHub Copilot** và **Antigravity AI Assistant**), cho phép đội ngũ tinh gọn gồm 6 kỹ sư đạt được năng suất tương đương một đội ngũ truyền thống từ 12-15 thành viên.



1.1 Vai trò của AI trong các Giai đoạn SDLC

Giai đoạn SDLC	Vai trò Lập trình viên	Hỗ trợ từ Trợ lý AI (Antigravity / Copilot)	Mức Tăng tốc
Yêu cầu & Đặc tả	Định nghĩa User Story & AC	Tự động tạo kịch bản Acceptance Test & phân rã WBS	2.5x
Kiến trúc & CSDL	Thiết kế ERD & API Contract	Tạo Prisma Schema, Script DB Migration & DTOs	3.0x
Lập trình Backend	Cài đặt nghiệp vụ cốt lõi	Sinh mã nguồn mẫu (Boilerplate) NestJS Controller, Service, Repository	3.5x
Phát triển Frontend	Thiết kế & thi công UI/UX	Sinh Component React/Next.js Tailwind & Hook quản lý trạng thái	3.0x
Kiểm thử & Sửa lỗi	Kiểm thử E2E & UAT	Tự động viết Jest/Vitest unit test, phân tích stack trace	4.0x
Tài liệu hóa	Đánh giá & xuất bản	Tự động sinh tài liệu OpenAPI/Swagger & tài liệu kỹ thuật	5.0x

2. CÁC KỊCH BẢN SINH MÃ NGUỒN BẰNG AI TRONG THỰC TẾ

2.1 Kịch bản 1: Sinh Module Catalog Backend NestJS

- Prompt cung cấp cho AI:

"Tạo NestJS Controller và Service cho module Catalog trong dự án LIBIF. Yêu cầu: Sử dụng Prisma ORM để truy vấn sách, hỗ trợ phân trang (page, limit), lọc theo categoryId và tags, và tìm kiếm toàn văn (full-text search) theo title/author. Tích hợp class-validator DTOs và các chú thích (annotations) Swagger."

- Mã nguồn do AI sinh ra (Đoạn mã NestJS Service):

```
// src/modules/catalog/catalog.service.ts
import { Injectable } from '@nestjs/common';
import { PrismaService } from '../../prisma/prisma.service';
import { FilterCatalogDto } from '../dto/filter-catalog.dto';

@Injectable()
export class CatalogService {
  constructor(private readonly prisma: PrismaService) {}

  async findAll(query: FilterCatalogDto) {
    const { page = 1, limit = 10, categoryId, search, tags } = query;
    const skip = (page - 1) * limit;

    const where: any = {};
    if (categoryId) where.categoryId = categoryId;
    if (tags && tags.length > 0) where.tags = { hasSome: tags };
    if (search) {
      where.OR = [
        { title: { contains: search, mode: 'insensitive' } },
        { author: { contains: search, mode: 'insensitive' } },
      ];
    }

    const [items, total] = await Promise.all([
      this.prisma.book.findMany({
        where,
        skip,
        take: limit,
        orderBy: { createdAt: 'desc' },
      }),
      this.prisma.book.count({ where }),
    ]);

    return {
      data: items,
      meta: { total, page, limit, totalPages: Math.ceil(total / limit) },
    };
  }
}
```

2.2 Kịch bản 2: Sinh Component Thanh Công cụ Reader DRM Next.js

- Prompt cung cấp cho AI:

"Tạo một React component trong Next.js 14 (TypeScript, TailwindCSS) cho thanh công cụ điều khiển trình đọc Reader Controls Toolbar. Bao gồm các nút Phóng to/Thu nhỏ (Zoom In/Out), Chuyển trang Trước/Sau (Previous/Next), hiển thị số trang hiện tại, và nút bật/tắt Toàn màn hình (Fullscreen)."

- Mã nguồn do AI sinh ra (Đoạn mã React Component):

```
// src/components/reader/ReaderToolbar.tsx
'use client';
import React from 'react';
import { ZoomIn, ZoomOut, ChevronLeft, ChevronRight, Maximize } from 'lucide-react';

interface ReaderToolbarProps {
  currentPage: number;
  totalPages: number;
  zoom: number;
  onPageChange: (page: number) => void;
  onZoomChange: (zoom: number) => void;
  onToggleFullscreen: () => void;
}

export const ReaderToolbar: React.FC<ReaderToolbarProps> = ({
  currentPage, totalPages, zoom, onPageChange, onZoomChange, onToggleFullscreen
}) => {
  return (
    <div className="flex items-center justify-between bg-slate-900 text-white px-6 py-3 s
      <div className="flex items-center space-x-3">
        <button onClick={() => onPageChange(Math.max(1, currentPage - 1))} className="p-2
          <ChevronLeft className="w-5 h-5" />
        </button>
        <span className="text-sm font-medium">Page {currentPage} / {totalPages}</span>
        <button onClick={() => onPageChange(Math.min(totalPages, currentPage + 1))} class
          <ChevronRight className="w-5 h-5" />
        </button>
      </div>
      <div className="flex items-center space-x-4">
        <button onClick={() => onZoomChange(Math.max(0.5, zoom - 0.1))} className="p-2 ho
          <ZoomOut className="w-5 h-5" />
        </button>
        <span className="text-sm font-medium">{Math.round(zoom * 100)}%</span>
        <button onClick={() => onZoomChange(Math.min(2.0, zoom + 0.1))} className="p-2 ho
          <ZoomIn className="w-5 h-5" />
        </button>
        <button onClick={onToggleFullscreen} className="p-2 hover:bg-slate-800 rounded bo
          <Maximize className="w-5 h-5" />
        </button>
      </div>
    </div>
  );
};
```

3. CONTAINER HÓA DOCKER & TỰ ĐỘNG HÓA CI/CD

3.1 Chiến lược Multi-Stage Dockerfile

Giảm kích thước Image từ **1.2GB xuống còn 145MB**, giúp tăng tốc độ triển khai và tăng cường bảo mật cho các container chạy trên môi trường sản xuất.

```
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json pnpm-lock.yaml ./
RUN npm install -g pnpm && pnpm install --frozen-lockfile
COPY . .
RUN pnpm build

FROM node:20-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production
RUN addgroup --system --gid 1001 nodejs && adduser --system --uid 1001 nestjs

COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/package.json ./package.json

USER nestjs
EXPOSE 3000
CMD ["node", "dist/main.js"]
```

3.2 Tuyển xử lý CI/CD GitHub Actions (`.github/workflows/deploy.yml`)

```
name: LIBIF CI/CD Pipeline

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  lint-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'pnpm'
      - name: Install Dependencies
        run: pnpm install
      - name: Run Linter
        run: pnpm lint
      - name: Run Unit Tests
        run: pnpm test:cov

  build-and-push-docker:
    needs: lint-and-test
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'
    steps:
      - uses: actions/checkout@v4
      - name: Login to DockerHub
        uses: docker/login-action@v3
        with:
          username: ${ secrets.DOCKERHUB_USERNAME }
          password: ${ secrets.DOCKERHUB_TOKEN }
      - name: Build & Push Docker Image
        uses: docker/build-push-action@v5
        with:
          context: .
          push: true
          tags: libif/backend:latest,libif/backend:${ github.sha }

  deploy-staging:
    needs: build-and-push-docker
    runs-on: ubuntu-latest
    steps:
      - name: Deploy to Cloud Server via SSH
        uses: appleboy/ssh-action@v1.0.0
        with:
```

```
host: ${ secrets.SERVER_HOST }}
username: ${ secrets.SERVER_USER }}
key: ${ secrets.SSH_PRIVATE_KEY }}
script: |
    cd /opt/libif
    docker compose pull
    docker compose up -d --remove-orphans
    docker system prune -f
```

4. TÓM TẮT HIỆU QUẢ PHƯƠNG PHÁP

Sự kết hợp giữa **Agile Scrum + Trợ lý AI + CI/CD Container hóa** mang lại:

- **Tốc độ:** Rút ngắn chu kỳ lập tính năng từ **3 tuần xuống chỉ còn 1 tuần**.
- **Chất lượng Mã nguồn:** 100% PRs vượt qua kiểm tra linter tự động và unit test với **Bao phủ Mã nguồn > 82%**.
- **Hiệu quả Triển khai:** Triển khai không gián đoạn (Zero-downtime deployment) hoàn tất trong **45 giây** thông qua Docker Compose.